Day 16 Evidence Workbooklet

List Mutation, Indexed Processing, and Exam 2 Readiness

Engineering practice to list state to indexed update to debug test to Exam 2 readiness

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/53		Mastery check	secure / developing / needs support	

Concrete engineering situation

The checkout desk now needs to correct and summarize stored charge evidence before Exam 2. Day 15 introduced lists, indexes, and list state. Day 16 turns that first exposure into assessment-ready evidence: trace list mutations, choose value loops or index loops, debug off-by-one errors, and route support before Exam 2.

Engineering task	Correct a stored list of checkout evidence and justify each update with trace and test evidence.
Computational move	Use list state, index positions, range(len(...)), mutation, and looped debugging to process stored evidence.
Python focus	list length, valid indexes, append, index update, loop over values, loop over indexes, range(len(...)), off-by-one boundary, final list state, and expected-vs-actual debugging.
Evidence to keep	index, old value, condition result, action, new value, list state after mutation, boundary test, skipped-item test, and support route.

Day 16 working rules

1. len(the list) gives the number of items, not the last index.
2. The last valid index is len(the list) - 1.
3. Use a value loop when you only need to read each value.
4. Use an index loop when you need to update a specific list item.
5. After a mutation, write the new list state before tracing the next pass.

Evidence map Manage your evidence blueprint

Part	Section	Evidence focus
1	Recover list length, index, and state	retrieve, state
2	Choose a value loop or an index loop	decide, explain
3	Trace indexed processing that writes back	index, mutation
4	Debug an off-by-one list loop	debug, boundary
5	Prepare an Exam 2 trace ledger	exam2, test
6	Transfer and support route before Exam 2	transfer, reflect

1 Recover list length, index, and state

retrieve state

Start by recovering the Day 15 evidence. Do not mutate a list until you can name its length, valid indexes, and current state.

```
charges = [82, 77, 91]
charges.append(80)
charges[1] = 88
print(charges)
```

Pts.	Prompt	My evidence / response
1	What is the list state before append?	
1	What is the list state after append?	
1	Which item changes when charges[1] = 88 runs?	
1	What final list prints?	
1	What is the last valid index after append?	

Support route

If length is 4, valid indexes are 0, 1, 2, and 3.

2 Choose a value loop or an index loop

decide explain

Choose the loop shape from the job. Reading a value is different from writing back into a list position.

```
charges = [82, 77, 91, 69]
ready_count = 0

for charge in charges:
    if charge >= 80:
        ready_count = ready_count + 1

print(ready_count)
```

Pts.	Prompt	My evidence / response
2	What values does charge take in order?	
2	Why is a value loop enough for counting ready kits?	
2	What would this loop not be able to change in the list?	
2	When would an index loop be more appropriate?	
2	Write one sentence that separates read-only counting from updating list state.	

3 Trace indexed processing that writes back

index mutation

An index loop is useful when the program needs to update the list item at that position. Trace old value, action, new value, and list state.

```
charges = [82, 77, 91, 69]
fix_count = 0

for index in range(len(charges)):
    if charges[index] < 80:
        charges[index] = 80
        fix_count = fix_count + 1

print(charges)
print(fix_count)
```

Pts.	Prompt	My evidence / response
2	Pass index 0: old value, condition result, action, list state after pass.	
2	Pass index 1: old value, condition result, action, list state after pass.	
2	Pass index 2: old value, condition result, action, list state after pass.	
2	Pass index 3: old value, condition result, action, list state after pass.	
2	What final list and fix_count print?	

4 Debug an off-by-one list loop

debug boundary

The intended task is to check every item and raise any low charge to 80. This version skips evidence.

```
charges = [72, 81, 77, 90]
for index in range(1, len(charges)):
    if charges[index] < 80:
        charges[index] = 80
print(charges)
```

Pts.	Prompt	My evidence / response
2	Which charge value is skipped completely?	
2	Why does range(1, len(charges)) skip it?	
2	What wrong final list prints?	
2	Write the corrected range call.	
2	Name a first-item test that would catch this bug.	

Common mistake to avoid

Starting an index loop at 1 skips index 0. That may hide a failing first item.

5 Prepare an Exam 2 trace ledger

exam2 test

Exam 2 can combine loops, conditions, and lists. Build the ledger before writing a final answer.

Pts.	Prompt	My evidence / response
2	For a list-processing trace, what columns should your ledger include?	
2	What boundary test checks the first item?	
2	What boundary test checks the last item?	
2	What expected-vs-actual evidence would show a skipped-last-item bug?	
2	Which earlier skill returns here: for/range, while termination, break/continue, nested loop, list state, or debug/test?	

6 Transfer and support route before Exam 2

transfer reflect

Close Day 16 by deciding what evidence you can use independently and what support route you need before Exam 2.

Pts.	Prompt	My evidence / response
4	Write a short explanation of when to use a value loop and when to use an index loop.	
2	Circle your support route: length/index / list state / mutation / value loop / index loop / off-by-one / debug-test. Name one next action.	
2	Name one thing Exam 2 should not require beyond this bridge boundary.	

Support route

Exam 2 should assess loops, list state, mutation, indexed processing, and debugging within the scaffolded bridge boundary. It should not require dictionaries, files, pandas, classes, recursion, or unscaffolded programming claims.